

# Entwine: Coordinating Tiled Computation and Fine-Grained Communication across GPUs

Kai Ma<sup>1,2,3</sup> Quanfeng Lv<sup>1,2,3</sup> Jingguo Ge<sup>1,2,3,\*</sup> Bowei Dai<sup>4,\*</sup> Kefan Ruan<sup>1,2,3</sup>

<sup>1</sup>State Key Laboratory of Cyberspace Security Defense

<sup>2</sup>Institute of Information Engineering, Chinese Academy of Sciences

<sup>3</sup>University of Chinese Academy of Sciences

<sup>4</sup>Institute of Microelectronics, Chinese Academy of Sciences

makai@iie.ac.cn lvquanfeng@iie.ac.cn gejingguo@iie.ac.cn

daibowei@ime.ac.cn ruankefan@iie.ac.cn

\*Corresponding authors

## Abstract

Modern high-performance GPU computations partition tensors into tiles to exploit data reuse and parallelism. Individual tile computations complete earlier than the full tensor computation, creating opportunities to overlap computation and communication. However, a mismatch between computation and communication progress can limit these opportunities. Communication stalls when no data is ready, and may lag when data arrives in bursts. Communication can also slow computation by consuming shared resources, offsetting the benefits of overlap.

We present Entwine, which coordinates tile computation order, fine-grained communication, and SM resource allocation to minimize overall completion time. Entwine reorders tile computation to produce data for communication at a more regular pace. Entwine couples this schedule with fine-grained SM-based communication to process tile results with low latency and low overhead. Since the communication kernel also consumes SM resources, Entwine coordinates their allocation to balance communication progress against computation slowdown. Across representative tensor-parallel LLM workloads, Entwine achieves a geomean speedup of  $1.232\times$  (up to  $1.433\times$ ) over cuBLAS+NCCL, and outperforms state-of-the-art overlap baselines by 3.1–9.8% in geomean. We will open-source our implementation upon publication.

## 1 Introduction

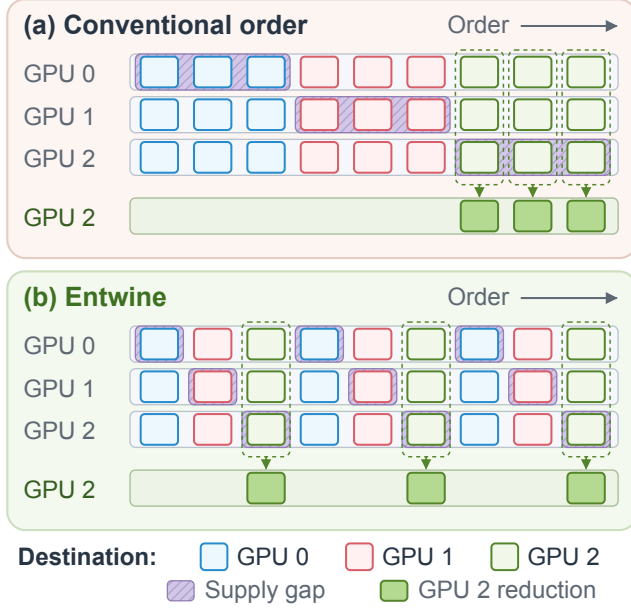
Large language models are often distributed across multiple GPUs to meet their memory and computational demands [16, 20, 43]. Tensor parallelism partitions the computation of individual layers across devices, but introduces communication to exchange and combine intermediate results [14, 35, 41]. This communication can account for a substantial fraction of execution time and limit the benefit of parallel computation [34]. Reducing communication overhead is therefore essential to efficient multi-GPU execution [1].

Overlapping computation and communication can hide this overhead, but data dependencies complicate their concurrent execution. When communication depends on the

output of a computation, conventional execution waits for the entire output before starting communication. High-performance matrix multiplication (GEMM) kernels, however, compute output in smaller regions, or tiles [37, 38], that finish at different times. These completed tiles provide opportunities to advance communication while the remaining output is still being computed.

Prior work enables dependent computation and communication to overlap through decomposition, signaling, and kernel fusion [29, 30, 39]. CoCoNet pipelines communication chunks with GEMM, while FlashOverlap releases completed waves or wave groups to library collectives [9, 12]. These approaches amortize communication overhead, but tiles completed early must wait for the rest of their communication group. FLUX enables tile-level transfers through GEMM epilogue fusion [2]. In this execution pattern, output transfers are issued as part of tile computation, coupling communication parallelism to the GEMM execution structure. This coupling restricts how independently the two activities can be scheduled and provisioned [45]. Comet instead assigns computation and communication to specialized blocks and profiles their resource split [42]. However, a resource split chosen for an entire execution may not match the changing balance between computation and communication demand within that execution. These execution choices constrain both when completed output can be communicated and how communication competes with subsequent computation.

These constraints lead to two coupled challenges. First, computation must sustain the supply of tiles for communication, and communication must process those tiles promptly. The production order must therefore be coordinated with tile-level communication, so that early outputs do not wait for unrelated producer blocks. Second, creating and exploiting these opportunities must reduce end-to-end latency despite the cost imposed on computation. Reordering can reduce data reuse, while concurrent transfers and reductions compete for SM resources and memory bandwidth. Both can slow subsequent tile production. A more regular supply or a shorter communication tail is therefore valuable only when



**Figure 1.** Tile computation order and its effect on communication in GEMM-ReduceScatter. Colors denote destination GPUs; purple bands mark locally consumed tiles. Interleaving disperses locally retained tiles, producing communication data at a more regular pace. Dashed lines connect contributions to GPU 2’s reductions.

its benefit exceeds the accompanying computation slowdown.

We present ENTWINE, which coordinates tile production, tile-level communication, and shared-SM resources to overlap computation with collective communication. We realize this design for GEMM-ReduceScatter as a concrete and widely used instance. ENTWINE interleaves tile computation across output partitions, distributing locally retained tiles among those requiring inter-GPU transfer (Figure 1, bottom). This order supplies communication inputs at a more regular pace. To exploit this supply promptly, ENTWINE releases and communicates output tiles individually. Each tile reduction requires the corresponding contributions from all ranks. Its input dependencies do not extend to a larger group of output tiles.

ENTWINE coordinates this tile-level execution in a shared SM pool to balance communication progress against computation slowdown. One computation kernel and one communication kernel execute concurrently, without a separate launch for each tile. As blocks finish, their resources become available to pending work from either kernel. A communication budget bounds the number of concurrent communication blocks, balancing the processing of completed tiles against the production of subsequent ones. We jointly select computation and communication configurations through offline profiling of end-to-end latency. This selection accounts

for both the overlap gained and the computation slowdown, instead of targeting GEMM throughput or the communication tail in isolation.

This paper makes the following contributions:

- We present ENTWINE, which coordinates tile production, tile-level communication, and shared-SM execution to reduce overall completion time.
- We introduce an interleaved production order coupled with independent tile-level communication to sustain data supply and process completed output without cross-block assembly waits.
- We coordinate computation and communication in a shared SM pool through a communication budget selected for end-to-end latency. We further relate achieved GEMM throughput to the average communication bandwidth needed to keep pace with production.
- We evaluate Entwine across tensor-parallel workloads and GPU counts. On our main suite, it achieves a  $1.232\times$  geomean speedup over cuBLAS+NCCL, with a maximum of  $1.433\times$ , and outperforms state-of-the-art overlap baselines by 3.1–9.8% in geomean. Ablations validate the effectiveness of each individual mechanism.

## 2 Background and Motivation

### 2.1 GEMM-ReduceScatter Execution

We use a row-parallel GEMM followed by ReduceScatter to illustrate the dependencies between computation and communication [12, 40]. With  $W$  GPUs, each rank holds a slice of the GEMM reduction dimension and computes a partial contribution to the output. ReduceScatter sums these contributions and assigns a distinct row partition of the result to each rank [5, 13]. Each rank therefore produces contributions both to its own output partition, which are consumed locally, and to other partitions, which require inter-GPU transfer.

We call the computation that generates output tiles the *producer* and the communication that processes them the *consumer*. The producer kernel executes as a grid of independently scheduled thread blocks [22]. A *tile* is an independently processable output region. In the producers studied here, each tile is computed by one block, while a block may cover multiple tiles. Output becomes available incrementally as blocks finish, each completing only its rank’s contributions to the tiles it covers. Reducing an output tile requires the corresponding contributions from all ranks, but has no data dependence on unrelated output tiles.

### 2.2 Motivation

In sequential execution, ReduceScatter starts only after every producer block has completed. Across the workloads in our

evaluation suite (§5.1), this organization leaves communication exposed for 11.0–40.2% of end-to-end latency, with a median of 23.3%.

★ **Insight 1: Sustaining the Supply of Communication-Ready Tiles.** *Effective overlap requires computation to continuously produce communication-ready tiles and communication to process them promptly.*

Computation progress does not always supply new data for inter-GPU transfer. Under a traversal that groups tiles by output partition, a producer computes a contiguous run of tiles for its own partition (Figure 1, top). During this run, the GEMM continues to make progress, but the completed tiles are consumed locally rather than transferred to peer GPUs. This creates a gap in the production of communication data on that GPU, reducing the portion of its remaining computation that can overlap with inter-GPU transfer.

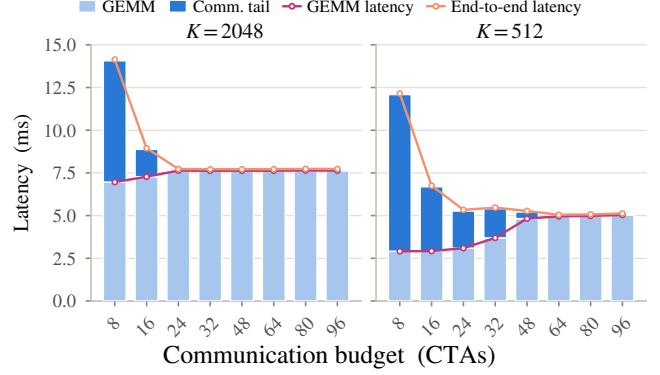
Waiting for unrelated output can limit the overlap gained from earlier tile production. The communication unit need not match the producer’s tile. FlashOverlap, for example, groups one or more execution waves and triggers a library collective after the group completes [9]. Grouping forms larger messages for better bandwidth utilization. However, a tile whose required contributions are available may still wait for unrelated output in the group. This waiting shortens the computation window available for overlapping the tile’s communication. Tile-level signals alone cannot eliminate this wait; communication must also process tiles independently.

★ **Insight 2: Balancing Overlap against Computation Slowdown.** *Earlier communication improves overall performance only when its benefit outweighs the additional cost imposed on computation.*

Creating and exploiting overlap opportunities can also reduce computational efficiency. Changing the production order may reduce data reuse (§5), while concurrent transfers and reductions consume SM resources and memory bandwidth also used by computation [6, 27]. Both effects can slow the production of subsequent tiles.

To examine the resource tradeoff, we vary a communication *budget*, defined as a cap on the number of concurrently admitted communication blocks. Figure 2 shows the response of two GEMM–ReduceScatter instances with the same output shape but different reduction dimensions  $K$ , yielding a longer and a shorter computation window. Within each panel, the production order, tile granularity, and data path are fixed, isolating the effect of communication concurrency.

With the longer window ( $K = 2048$ ), increasing the budget initially shortens the communication tail. The reduction in the tail outweighs the increase in GEMM time, so end-to-end latency falls before reaching a broad near-optimal region. With the shorter window ( $K = 512$ ), increasing the budget



**Figure 2.** Effect of the communication budget on end-to-end GEMM–ReduceScatter latency for a longer ( $K = 2048$ , left) and a shorter ( $K = 512$ , right) computation window. Bars separate GEMM time from the exposed communication tail; curves report GEMM time and end-to-end latency. Within each panel, the tile granularity, producer order, and data path are fixed.

helps only until the added GEMM delay offsets the reduction in the communication tail. End-to-end latency reaches an interior minimum, beyond which additional communication capacity provides no benefit. The appropriate allocation is therefore determined by end-to-end latency, not by the shortest communication tail.

Section 3 presents how ENTWINE coordinates these choices to reduce end-to-end latency.

### 3 Design

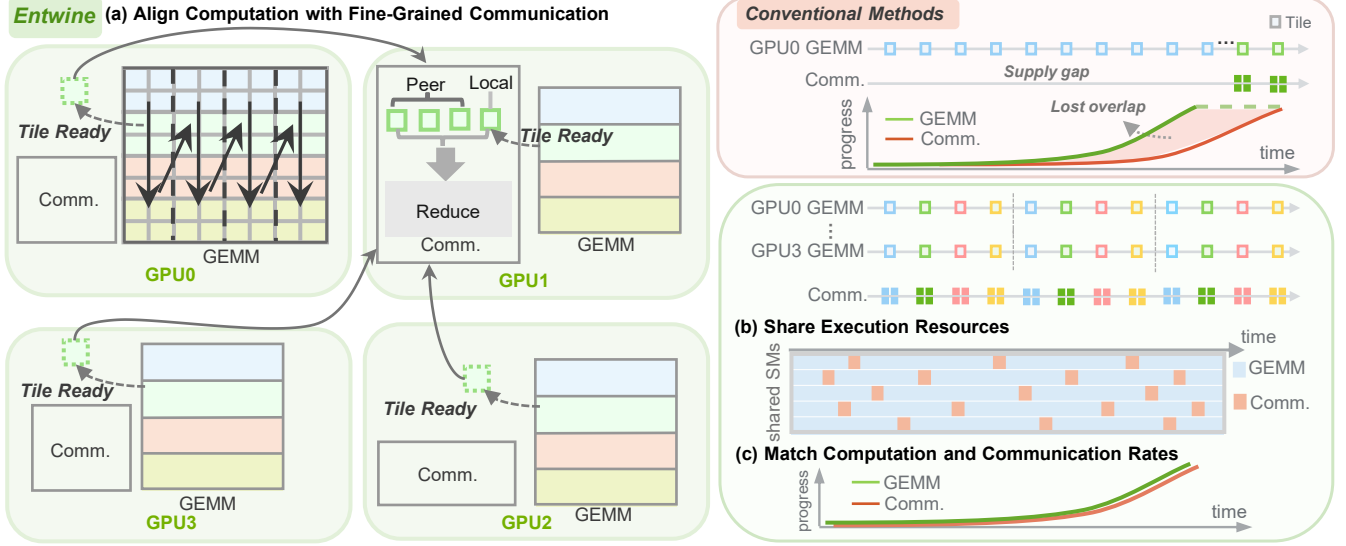
ENTWINE coordinates tiled computation and communication to minimize end-to-end latency. Computation determines when communication can begin for each tile. Communication shares resources with computation and can slow the production of subsequent tiles. We therefore choose the production order, communication granularity, and kernel configurations together.

We present the design for GEMM–ReduceScatter. Section 3.1 describes the tile order and communication protocol. Section 3.2 explains how we select kernel configurations under resource sharing. Section 3.3 analyzes the communication bandwidth required at a given configuration. Figure 3 summarizes the design, and Figure 4 illustrates its execution.

#### 3.1 Coordinating Tiled Computation and Communication

All ranks compute partial outputs with the same dimensions. The final output is partitioned across destination ranks. To form output tile  $O[d, t]$ , destination rank  $d$  reduces the corresponding contributions  $V[r, d, t]$  from all source ranks  $r$ .

**Interleaved production.** With a contiguous traversal, a producer can spend an extended interval computing only its



**Figure 3.** Overview of ENTWINE for GEMM-ReduceScatter on four GPUs. (a) Four colors denote output partitions. For each producer, one partition is retained locally; interleaving distributes its tiles among those requiring remote transfer. Arrows trace peer contributions to GPU 1’s reduction. (b) Computation (blue) and communication (orange) blocks share an SM pool; retiring blocks release capacity for pending work from either kernel. (c) Cumulative computation and communication progress under coordinated execution. The upper-right conventional comparison marks an interval of *lost overlap*.

locally retained partition, supplying no new data for inter-GPU transfer. ENTWINE cycles through the output partitions in logical block order, as illustrated in Figure 4(a). Only one partition is local to each source rank. In the logical block order, blocks producing local contributions are interspersed with blocks producing contributions for peers.

For  $W$  equal, block-aligned output partitions, let  $p$  denote a producer block’s logical position, numbered from zero. Its destination rank  $d$  and within-partition block index  $\ell$  are

$$d = p \bmod W, \quad \ell = \lfloor p/W \rfloor. \quad (1)$$

On rank  $r$ , blocks with  $d = r$  produce contributions to the local partition; the others produce contributions for peers. All ranks use the same mapping, placing contributions to the same tile at corresponding logical positions. The destination’s task order can therefore follow a common production order across ranks. The mapping specifies logical block order; physical completion remains subject to GPU scheduling (§2.1).

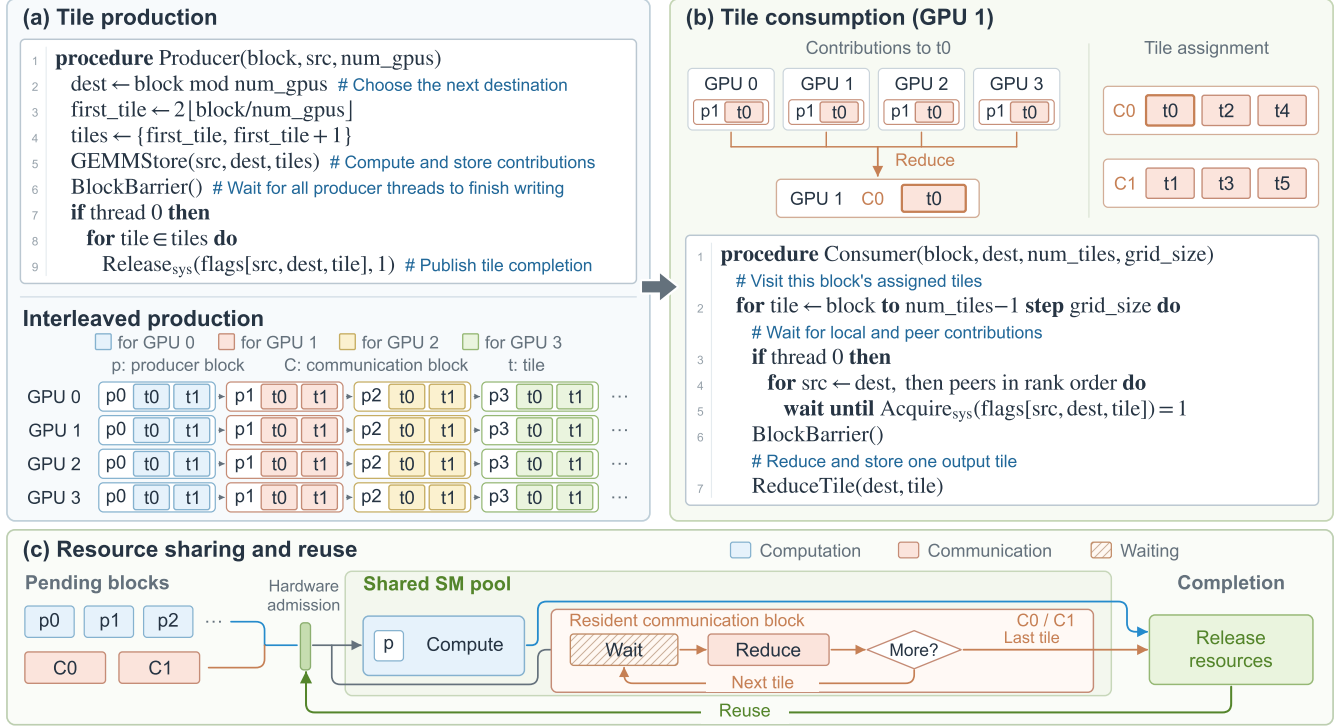
**Independent tile communication.** Each communication task reduces one output tile. The task depends on one producer block per rank, which supplies that rank’s contribution (§5.3.2). It need not wait for the rest of the destination partition. The upper-left part of Figure 4(b) illustrates this dependency for tile  $t_0$  in GPU 1’s partition. Each rank’s block  $p = 1$  supplies a contribution to that tile. GPU 1’s communication block  $C_0$  reduces the four contributions, including the one produced locally.

The producer completes its output stores before publishing a completion signal  $F[r, d, t]$  for each contribution. Before reading a tile’s inputs, the consumer block waits for the signals from all ranks (Figure 4(a,b)). It then reduces the tile. This protocol ensures that reads follow the stores of all required contributions. Section 4.2 describes the memory ordering that enforces it.

A computation block groups output elements to exploit data reuse and may cover several tiles. These tiles retain separate reduction tasks, which can be distributed across consumer blocks even when their inputs are produced together. This separates communication granularity from the GEMM block shape, allowing computation to exploit reuse across multiple tiles.

**Matching task order.** One communication kernel processes all tile reductions for the local output partition. Each tile is assigned to one consumer block, and each block executes a sequence of tasks. It waits for and reduces its current tile before advancing to the next. Blocks advance independently, without separate kernel launches for successive tiles.

Within each output partition, number the  $Q$  tiles from zero in producer traversal order. With  $x$  consumer blocks, block  $j$  processes tiles  $j, j + x, j + 2x, \dots$  up to the last index below  $Q$ . Each sequence follows the producer’s logical traversal order, so a consumer checks tiles in the order of their assigned producer blocks. It still waits for its current tile if later producer blocks finish first.



**Figure 4.** Tile-level execution in ENTWINE. The example uses four GPUs, two tiles per producer block, and two communication blocks per destination. (a) Producer procedure and interleaved production. (b) Tile consumption, dependencies, and tile assignment; the outlined  $t_0$  is expanded at upper left. Each communication block processes its assigned tiles in increasing tile-index order. (c) Block lifecycles and resource reuse. The arrow from (a) to (b) denotes per-tile dependencies. Tile colors denote destinations in (a,b); colors in (c) denote kernel roles.

The upper-right part of Figure 4(b) shows this assignment with  $x = 2$ . Block  $C_0$  processes  $t_0, t_2, t_4$  in sequence, while  $C_1$  processes  $t_1, t_3, t_5$ . In this example, each producer block covers two tiles. On every rank, block  $p = 1$  produces contributions to  $t_0$  and  $t_1$ . By Equation (1), the next visits to GPU 1’s partition use blocks  $p = 5$  and  $p = 9$ , which supply the next two pairs. Each consumer thus takes one tile from each pair.

Changing  $x$  redistributes the same tile tasks among consumer blocks. A smaller grid gives each block a longer sequence without combining the dependencies of successive tiles. Once its current tile’s inputs are available, a block can reduce that tile while later tiles are still being computed. The GEMM block shape determines which tiles are produced together;  $x$  determines how their reductions are distributed across blocks.

**Data reuse.** Interleaving across output partitions may cause successive producer blocks to access different row panels of  $A$ , reducing opportunities for reuse. The choice of GEMM block shape therefore affects the tradeoff between data reuse and the overlap enabled by interleaving (§5.3.1).

We assess this tradeoff under concurrent execution, where both kernels also compete for resources.

### 3.2 Optimization under Resource Sharing

ENTWINE executes one computation kernel and one communication kernel concurrently on each GPU. Separate kernels allow communication concurrency to vary without changing the GEMM block shape. Their performance remains coupled through shared SM resources and memory bandwidth [26, 36].

**Communication budget.** The communication budget  $x$  is the grid size of the communication kernel, capped by the number of tiles in the rank’s output partition. The thread count  $b$  controls parallelism within each communication block and affects its resource requirements. Together,  $x$  and  $b$  determine how much communication work can compete with GEMM.

Communication blocks retain their SM resources while waiting for inputs. The budget must therefore leave capacity for computation to produce those inputs (§4.3). The grid size stays constant throughout an invocation, while the number of resident communication blocks changes as blocks are

admitted and complete. No fixed set of SMs is reserved for communication.

**Resource reuse.** Blocks from both kernels share an SM pool (Figure 4(c)). The hardware scheduler admits pending work from either kernel as resources become available [22]. A computation block releases its resources after computing its output region. A communication block retains its resources across successive tasks and releases them after its last reduction. For example,  $C_0$  keeps its allocation while waiting for  $t_2$  and releases it after reducing  $t_4$ . Pending computation can then use the released capacity while other communication blocks remain active.

**End-to-end optimization.** Larger budgets allow more communication blocks to make progress, but may slow computation through competition for SM resources and memory bandwidth (Figure 2). The global communication tail is the interval between the last computation kernel finishing and all communication completing. Increasing concurrency can shorten this tail while extending the computation window.

The GEMM block shape also affects this tradeoff through data reuse and the number of tiles produced together. We therefore evaluate GEMM configurations together with the communication budget  $x$  and thread count  $b$ . ENTWINE selects the combination with the lowest end-to-end latency under concurrent execution. This criterion accounts for both communication progress and GEMM slowdown, which standalone kernel measurements do not capture (§5.4.2, §5.5.1). Section 4.3 describes the offline profiling procedure.

### 3.3 Communication Bandwidth Analysis

We now relate achieved GEMM throughput to the average communication bandwidth required during GEMM execution. This analysis explains the bandwidth requirements of configurations selected by end-to-end latency (§3.2).

Consider one rank computing an  $M \times N$  partial output over local reduction dimension  $K$ . Finalizing its  $1/W$  output partition reads contributions from  $W - 1$  peers, with total remote-read volume

$$V_{\text{comm}} = \frac{W - 1}{W} MNs, \quad (2)$$

where  $s$  is the size of a stored contribution element in bytes. Let  $\Phi(x, K)$  denote this rank’s achieved GEMM throughput during concurrent execution, measured in FLOP/s. It includes the slowdown caused by communication. Its dependence on output shape, datatype, rank count, GEMM configuration, and communication thread count is implicit. The rank’s GEMM latency is

$$T_G(x, K) = \frac{2MNK}{\Phi(x, K)}. \quad (3)$$

The average bandwidth required to process the remote-read volume within this window is

$$B_{\text{req}}(x, K) = \frac{V_{\text{comm}}}{T_G(x, K)} = \frac{W - 1}{W} \frac{s\Phi(x, K)}{2K}. \quad (4)$$

At fixed output dimensions, rank count, and datatype, increasing  $K$  adds computation without adding communication volume. The required bandwidth falls when GEMM throughput grows more slowly than  $K$ . Conversely, a shorter computation window requires higher average bandwidth for the same volume. The explicit factors  $M$  and  $N$  cancel, but output shape still affects the required bandwidth through GEMM throughput.

The budget  $x$  also affects the required bandwidth through GEMM throughput. If additional communication blocks slow GEMM, the computation window lengthens and the same volume requires a lower average bandwidth. Lower required bandwidth can therefore reflect slower GEMM, even when the communication volume is unchanged. This is why configuration selection considers end-to-end latency.

Equation (4) averages over the full computation window. Different production orders have the same required average bandwidth when communication volume and GEMM latency are unchanged. However, each tile can be reduced only after its inputs become available. Late inputs leave less computation to overlap with that reduction. The tile order in §3.1 therefore matters even when the average bandwidth requirement is unchanged. Section 5.4.1 compares required and measured communication bandwidth, and §5.4.2 examines GEMM slowdown as the budget increases.

## 4 Implementation

### 4.1 Execution Path

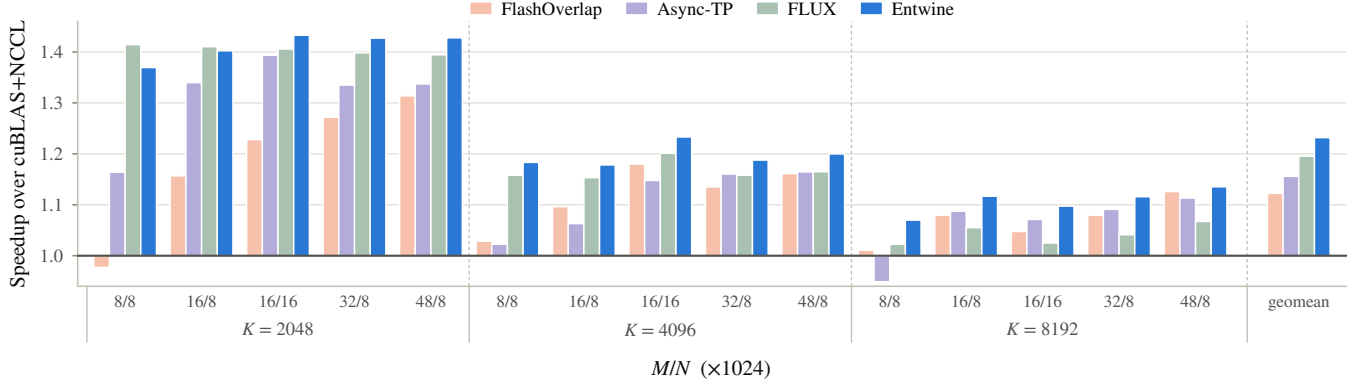
The prototype pairs a CUTLASS [23] GEMM with a CUDA reduction kernel for row-parallel GEMM–ReduceScatter (Figure 4). It targets NVIDIA SM80 GPUs within a single peer-accessible NVLink domain. Output partitions have equal row counts. Each partition’s row count and the output width are multiples of 128.

Each rank allocates its partial output and publication flags in a symmetric CUDA IPC heap. Contributions and flags occupy matching offsets in the heaps on all ranks. The reducer uses these offsets to read contributions directly from peer memory. Allocations are reused across invocations. Before each invocation, ranks reset their flags and synchronize.

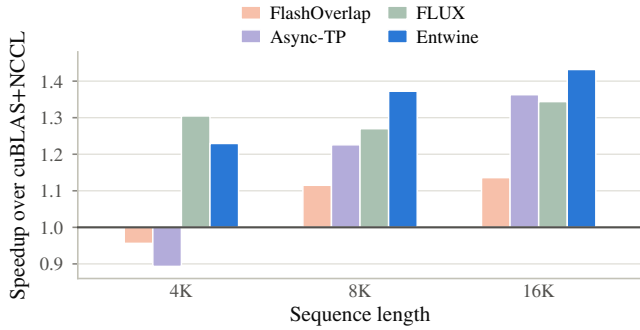
Each invocation launches the GEMM and reduction kernels on independent CUDA streams after a common start event. We capture their kernel launches in CUDA Graphs [22] to reduce repeated host-launch overhead.

### 4.2 Producer Traversal, Publication, and Reduction

Figure 4 presents the *Producer* and *Consumer* procedures, each executed cooperatively by one thread block on its source or destination rank. Their *block* arguments denote producer



**Figure 5.** End-to-end GEMM-ReduceScatter speedup over cuBLAS + NCCL. Workloads are grouped by local GEMM reduction dimension  $K$ ; the rightmost group reports the geomean.



**Figure 6.** Layer-level speedup over cuBLAS + NCCL for the Llama 3 70B attention-output GEMM, including residual addition and RMSNorm. Sequence lengths are global, before tensor-parallel partitioning.

position  $p$  and consumer index  $j$ , respectively. The parameters  $num\_gpus$ ,  $num\_tiles$ , and  $grid\_size$  denote  $W$ ,  $Q$ , and  $x$ . The producer retains the selected GEMM kernel’s tensor-core main loop. Figure 4(a) uses *GEMMStore* to denote computing and storing a producer block’s contributions. A custom CUTLASS thread-block swizzle implements the traversal in Equation (1) and Figure 4(a).

The reduction unit is a  $128 \times 128$  output tile. Producer blocks cover either  $128 \times 128$  or  $128 \times 256$  outputs. These configurations differ in data reuse under the interleaved order (§5.3.1). An epilogue visitor [4] publishes completion signals. The wider block publishes both tile flags after completing its output stores, as in Figure 4(a). The consumer uses the same tile size with either producer.

Producer threads synchronize after their output stores, so publication covers the writes of the entire thread block (CTA). Thread 0 then publishes the tile flags with system-scope release stores. The pseudocode writes  $F[r, d, t]$  as  $flags[src, dest, tile]$  and denotes system scope as  $sys$ .

For each tile in its assigned sequence, consumer thread 0 polls the local flag and then the peer flags using system-scope acquire loads. The local flag is necessary because the producer and consumer execute asynchronously on the same GPU.

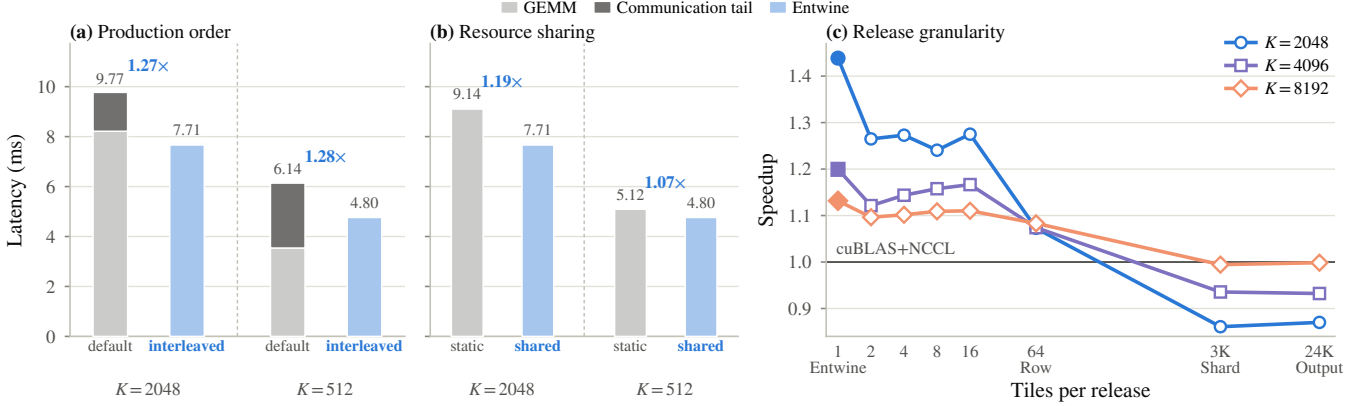
After observing all  $W$  flags for the current tile, thread 0 synchronizes with its CTA, extending the acquire ordering to all threads that read the tile. The release stores, acquire loads, and CTA barriers thus order these reads after the corresponding producer stores [17, 22].

The CTA cooperatively reduces the tile through *ReduceTile* in Figure 4(b). Each thread processes a disjoint subset of tile elements. For each element, it initializes an accumulator from the local contribution, adds peer values in rank order, and stores the result in the local output partition. Each output element therefore has a single writer. The implementation distributes packed FP16 pairs across threads in a strided loop. For two ranks, it groups these pairs into vector loads and stores.

### 4.3 Kernel Launch and Configuration

The GEMM kernel runs on a default-priority stream and the reduction kernel on a high-priority stream. Stream priority favors pending reducer work as SM capacity becomes available (Figure 4(c)). The hardware scheduler determines block admission, and running blocks continue to completion [22]. The default reducer grid is capped below the device’s SM count. This leaves capacity for GEMM blocks without reserving a fixed set of SMs, even while reducer CTAs wait for inputs.

For each matrix shape and rank count, offline profiling evaluates GEMM kernels and reducer grid sizes with 128, 256, or 512 threads per reducer block. We select the combination with the lowest end-to-end latency under concurrent execution (§3.2).



**Figure 7.** Effects of computation order, resource sharing, and release granularity. Panels (a) and (b) show end-to-end latency with the communication tail highlighted. Panel (c) reports end-to-end speedup as the release unit grows from 1 tile to the whole output, with a fixed one-tile reduction task. Each result includes the synchronization required by its release policy.

## 5 Evaluation

We evaluate whether ENTWINE’s coordination of tiled computation and fine-grained communication reduces end-to-end latency. We compare against a sequential reference and three existing overlap approaches at both operator and layer levels. Controlled ablations examine how computation order, release granularity, and resource sharing affect this benefit, and budget sweeps characterize the tradeoff between communication progress and computation slowdown.

### 5.1 Experimental Setup

**Testbed.** Experiments use 2, 4, or 8 NVIDIA A800-SXM4-80GB GPUs on one NVLink-connected node, with 8 as the default. We use CUDA 12.1, cuBLAS 12.1.3.1, NCCL 2.21.5, CUTLASS at commit 08185b9c, FlashOverlap at commit 38fe6a3, Async-TP from PyTorch 2.5.1+cu121 [28], and FLUX at commit ffb34a73.

**Workloads.** Two workload suites cover different ratios of computation to communication and different LLM GEMM shapes (Table 1). The main suite uses the MLP output widths of Llama 3 70B and Llama 3.1 405B [19] and covers 15 workloads spanning a  $6\times$  range in output size  $MN$  and a  $4\times$  range in per-rank reduction dimension  $K$ . At a fixed tensor-parallel width and datatype, increasing  $MN$  scales both GEMM work and ReduceScatter volume, whereas increasing  $K$  adds computation without adding communication. Equal-volume outputs with different aspect ratios separate output geometry from the amount of work.

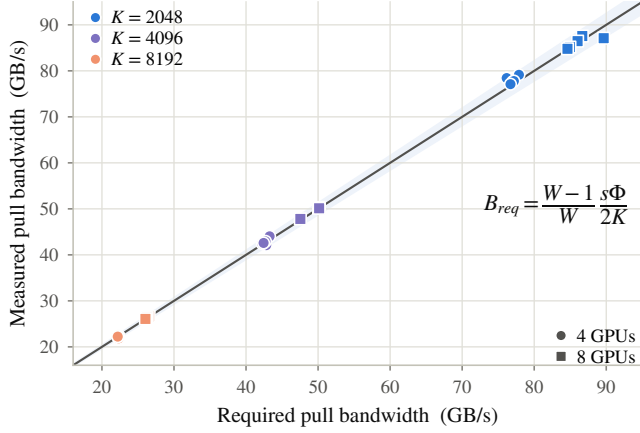
The model-derived suite includes the attention-output and MLP-output GEMMs of Llama 3 8B and 70B [19] and the MLP-output GEMM of Qwen2.5-72B [32]. Five logical activation sizes per GEMM yield 25 workloads. These cover shorter computation windows, complementing the main suite in the operating-range analysis (§5.5.1).

**Table 1.** Evaluation workloads on 8 GPUs.  $K$  is the per-rank reduction dimension; all methods pad model-derived cases with logical  $M = 512$  to physical  $M = 1024$ .

| Main operator suite — 15 workloads         |                                 |
|--------------------------------------------|---------------------------------|
| Llama 3 70B width, $N = 8192$              | $M = 8192, 16384, 32768, 49152$ |
| Llama 3.1 405B width, $N = 16384$          | $M = 16384$                     |
| Per-rank reduction $K$                     | 2048, 4096, 8192                |
| Model-derived suite at TP=8 — 25 workloads |                                 |
| GEMM                                       | $(N, K)$                        |
| Llama 3 8B attn. output                    | (4096, 512)                     |
| Llama 3 8B MLP output                      | (4096, 1792)                    |
| Llama 3 70B attn. output                   | (8192, 1024)                    |
| Llama 3 70B MLP output                     | (8192, 3584)                    |
| Qwen2.5-72B MLP output                     | (8192, 3696)                    |
| Envelope sweep, logical $M$                | 512, 1024, 2048, 4096, 8192     |

**Baselines.** The sequential reference, *cuBLAS + NCCL*, completes the GEMM before invoking NCCL ReduceScatter [21, 24]. *FlashOverlap* starts an asynchronous collective after releasing each completed wave group [9]. *Async-TP* pipelines ReduceScatter behind chunked GEMM execution through PyTorch’s symmetric-memory operator [8]. *FLUX* integrates reduction into the GEMM epilogue and autotunes tiling [2]. We use each method’s native implementation and recommended workload-specific tuning procedure.

**Protocol.** Unless stated otherwise, speedups are relative to cuBLAS + NCCL on the same workload, and geomeans weight workloads equally. We time 200 iterations with CUDA events after 120 warmups and report the maximum median latency across ranks. We validate outputs against an unfused reference with absolute tolerance 0.1 and relative tolerance 0.05.



**Figure 8.** Measured communication bandwidth versus average demand computed from the measured computation window (Equation (4)). Colors denote local  $K$ ; markers denote tensor-parallel width. The diagonal indicates equal measured and required bandwidth.

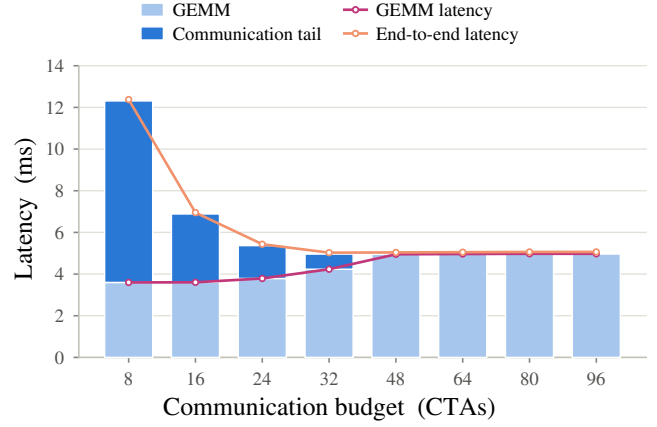
## 5.2 Overall Performance

**5.2.1 Operator-Level Performance.** Across the 15 main-suite workloads in Figure 5, ENTWINE achieves a  $1.232\times$  geomean speedup over cuBLAS + NCCL and a  $1.0185\times$  geomean speedup over the per-workload best among prior methods. It outperforms all three overlap approaches on 13/15 workloads. Geomean latency decreases relative to FlashOverlap, Async-TP, and FLUX by 8.88%, 6.17%, and 2.96%, respectively.

Across the suite, overlap efficiency—the ratio of uncontended GEMM time to end-to-end latency—reaches 0.946 in geomean, indicating that ENTWINE overlaps GEMM and ReduceScatter while keeping their combined latency close to that of standalone GEMM execution.

Speedup over sequential execution decreases as per-rank  $K$  increases at fixed output dimensions. Larger  $K$  extends the computation window without adding communication volume, so communication accounts for a smaller share of sequential execution. This reduces the speedup available from overlap. Output size also affects the comparison with FLUX: at  $K = 2048$ , FLUX outperforms ENTWINE on the two smallest outputs, which provide less computation to overlap reduction. §5.5.1 examines performance across a wider range of workloads.

**5.2.2 Layer-Level Performance.** To assess whether the overlap benefits carry over to a longer operator chain, we evaluate the Llama 3 70B attention output, including the GEMM and ReduceScatter followed by residual addition and RMSNorm. We vary sequence length while keeping the model and GEMM output width fixed (Figure 6). ENTWINE reduces latency relative to cuBLAS + NCCL, FlashOverlap, and Async-TP at all three sequence lengths. Relative to FLUX,



**Figure 9.** Effect of communication budget on latency at  $K = 768$ . Bars show computation windows and communication tails; curves show GEMM and end-to-end latency, including completion overhead. From 32 to 48 CTAs, computation slowdown offsets the shorter tail.

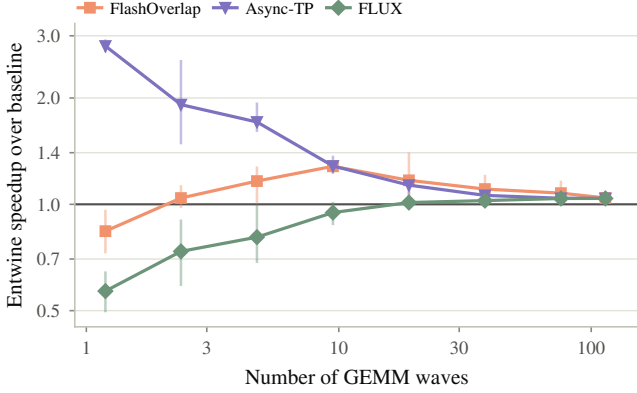
ENTWINE lowers layer-level latency by 8.1% and 6.5% at 8K and 16K, respectively, whereas FLUX has 5.8% lower latency at 4K. Longer sequences increase the GEMM wave count, providing a wider computation window over which reductions can progress concurrently. These results demonstrate that the benefits of ENTWINE’s fine-grained overlap extend to layer-level execution.

## 5.3 Mechanism Ablations

The ablations in Figure 7 isolate computation order, release granularity, and resource sharing. Each comparison changes one policy while holding the workload, tiling, data path, and consumer task assignment fixed. For policies with tunable communication concurrency, we sweep the same budget range and report each policy’s lowest end-to-end latency.

**5.3.1 Computation Order.** We compare ENTWINE’s interleaved mapping in Figure 7(a), which follows the consumer’s  $N$ -major order within each partition, with the producer kernel’s default  $M$ -major traversal. By coordinating tile production with reduction, ENTWINE lowers end-to-end latency in both workloads, including the case in which GEMM execution slows. At  $K = 512$ , the GEMM takes longer under interleaving, but the tail shrinks by 98.5%, yielding a net  $1.277\times$  end-to-end improvement. At  $K = 2048$ , both the computation window and the tail shrink, improving end-to-end performance by  $1.267\times$  with a 97.6% tail reduction. The default order leaves a substantial tail even at its best budget.

**5.3.2 Release Granularity.** To examine how release granularity affects overlap, we vary the number of tiles released together while keeping each reduction task fixed at one tile (Figure 7(c)). A group is released when its last tile finishes.



**Figure 10.** Speedup versus GEMM wave count. Marks show ENTWINE’s geomean speedup over each baseline at the same GEMM wave count; whiskers show the full workload range.

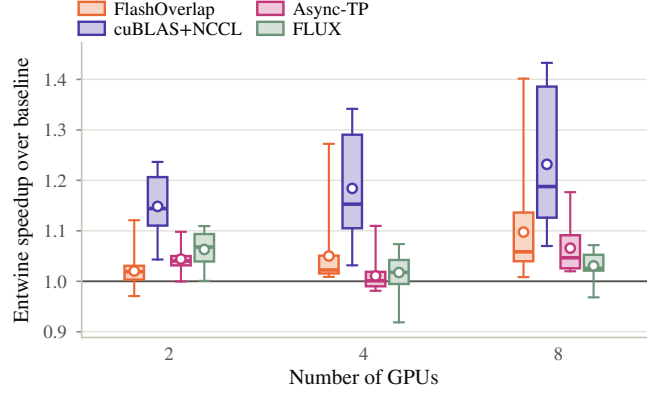
This comparison changes when reductions can begin while preserving the amount of work in each reduction task.

Tile-level release gives the highest speedup on all 3 workloads: 1.439 $\times$ , 1.199 $\times$ , and 1.132 $\times$  over the non-overlapped baseline. Small groups retain much of this benefit, whereas releasing a full output partition or the whole output brings performance to or below the baseline. Larger groups delay reductions on tiles completed earlier in the group, leaving less computation to overlap when those reductions begin. With the reduction tasks held fixed, ENTWINE’s tile-level release preserves overlap that is lost under partition- or output-level release.

**5.3.3 SM Resource Sharing.** Figure 7(b) compares shared-SM execution with static partitioning on the same two workloads as the computation-order ablation. Both policies use the same GEMM and reduction kernels, task order, and data path. We select the best communication budget for each policy independently. Static partitioning reserves a fixed subset of SMs for communication. Reduction blocks remain resident until GEMM finishes, even after completing their assigned reductions.<sup>1</sup> Shared-SM execution removes these restrictions and lets both kernels use the same SM pool.

At their selected budgets, both policies leave little communication tail. Shared-SM execution has a shorter computation window, improving end-to-end performance by 1.185 $\times$  at  $K = 2048$  and 1.066 $\times$  at  $K = 512$ . Under shared-SM execution, SMs become available to pending GEMM blocks after reductions finish. This allows ENTWINE to sustain communication progress with less GEMM slowdown than static partitioning.

<sup>1</sup>We enqueue reduction blocks before GEMM and allocate enough dynamic shared memory to allow exactly one reduction block per occupied SM.



**Figure 11.** ENTWINE speedup across tensor-parallel widths, with per-rank  $M$ ,  $N$ , and  $K$  fixed. Boxes show the interquartile range, lines the medians, whiskers the full range, and hollow circles the geomeans.

## 5.4 Communication Bandwidth and Concurrency

**5.4.1 Matching Communication to Computation.** At the selected budgets, measured communication bandwidth closely matches the average communication demand. We compute this demand from each rank’s measured computation window using Equation (4). Measured bandwidth is the remote-read volume divided by the communication kernel’s elapsed time, including waits for tile inputs. Across the 15 main-suite workloads at both 4 and 8 GPUs, required and measured bandwidth differ by at most 3.0%, with a median gap of 0.37% (Figure 8). This agreement indicates that ENTWINE’s selected configurations sustain communication at approximately the rate needed to process output during the computation window.

Within each  $(W, K)$  group, the 5 output shapes achieve similar GEMM throughput and cluster around a common bandwidth demand despite their different communication volumes. This clustering is consistent with Equation (4), in which the output dimensions cancel and shape affects demand through achieved GEMM throughput. At comparable throughput, larger  $K$  lowers the required rate by spreading communication over a longer computation window.

**5.4.2 Effect of Communication Concurrency.** A shorter communication tail does not necessarily reduce end-to-end latency. At  $K = 768$  (Figure 9), increasing the budget from 32 to 48 CTAs nearly eliminates the tail, but the accompanying computation slowdown offsets this gain, leaving total latency almost unchanged. The full budget sweep across eight values of  $K$  (Figure 13) shows the same pattern more broadly. For larger  $K$ , a larger budget initially shortens the tail with little effect on computation, and end-to-end latency reaches a broad plateau. For smaller  $K$ , more concurrent consumer work increasingly delays the producer and the output on which later reductions depend.

With all inputs available and no competing producer, communication bandwidth rises with the budget before reaching a plateau (Figure 12). Beyond that plateau, additional concurrency provides little communication capacity, while concurrent consumer work can still delay the producer through resource occupancy and memory traffic. Isolated bandwidth therefore does not capture the end-to-end effect of added communication concurrency. This supports selecting computation and communication configurations by end-to-end latency (§3.2), which reflects both communication progress and computation slowdown.

## 5.5 Sensitivity Analysis

Workload size and tensor-parallel width change the computation available for overlap and the communication that must complete within it.

**5.5.1 Operating Range.** Figure 10 combines both workload suites to characterize how ENTWINE’s overlap benefits vary with the available producer work. We group results by GEMM wave count, defined as the producer’s thread-block count divided by the number of SMs. The 25 model-derived workloads span 1.2–19 waves, and the 15 main-suite workloads span 19–114 waves, with the two ranges meeting at 19 waves. In addition to this variation in block count,  $K$  affects the bandwidth required during the computation window (§5.4.1).

ENTWINE’s geomean speedup over FlashOverlap crosses 1 in the 1.19–2.37-wave interval, and its geomean speedup over FLUX crosses 1 in the 9.48–18.96-wave interval. It remains ahead of Async-TP throughout the measured range. With few producer waves, little computation remains after contributions become available, limiting the time available for concurrent reductions. As the wave count grows, ENTWINE overlaps reductions with the remaining GEMM computation over a longer window (§5.3). The sequence-length trend in the layer-level results is consistent with this pattern (Figure 6).

**5.5.2 Tensor-Parallel Width.** Figure 11 compares 2, 4, and 8 GPUs while holding each workload’s per-rank GEMM dimensions fixed. This preserves local computation as the collective combines contributions from more ranks, increasing the remote data each rank must read (Equation (2)). ENTWINE outperforms all baselines in geomean at every tested width. Its geomean speedup over FlashOverlap grows from 1.0204 $\times$  at 2 GPUs to 1.0975 $\times$  at 8 GPUs, and over cuBLAS + NCCL from 1.1484 $\times$  to 1.2318 $\times$ . Across these widths, ENTWINE thus maintains its geomean performance advantage as communication demand grows relative to the fixed local GEMM workload.

## 6 Related Work

**Communication Granularity and Dependencies.** Decomposing dependent operations creates opportunities for overlap: communication on one part of a tensor can proceed while computation continues on another. CoCoNet jointly optimizes computation and communication through program transformations and joint kernel generation [12]. Wang et al. [40] and Domino [39] decompose dependent work into smaller operations that can be pipelined. Centauri partitions communication across primitives, device groups, and workloads [3]. SYNDICATE divides communication into motifs and jointly optimizes their scheduling and execution plans [18]. FLUX [2] and Punniyamurthy et al. [30] fuse computation with communication, while T3 [29] uses hardware to track output production and trigger communication. FlashOverlap communicates one or more completed execution waves at a time, using output reordering to form contiguous communication buffers [9]. Cui et al. [7] signal completion per tile but transfer larger segments spanning one or more full-width row bands in MoE. Both balance communication start time against transfer efficiency. ENTWINE instead communicates output tiles independently, avoiding waits for unrelated output in a larger group. TileLink [45], Triton-Distributed [44], and Syncopate [31] provide abstractions for expressing dependencies and generating overlapped execution.

**Communication-Aware Computation Ordering.** Computation order determines when communication inputs become available. Ordering policies depend on whether computation produces data for communication or consumes data received from other GPUs. CoCoNet computes MatMul output chunks in the order required by the collective [12]. FLUX uses tile swizzling to reduce conflicting writes across GPUs or align computation with incoming data [2]. Comet schedules computation on local inputs while remote inputs arrive [42]. For MoE return communication, Cui et al. prioritize output destined for remote GPUs [7]. ENTWINE reorders tile computation to supply communication data at a more regular pace, creating overlap opportunities throughout computation. TileLink exposes tile order as a design choice, including tradeoffs between data reuse and waiting for inputs [45]. Syncopate rewrites tile schedules to follow a communication plan [31].

**Resource Coordination for Overlap.** Concurrent communication competes with computation for SM resources and memory bandwidth. Comet assigns specialized thread blocks to computation and communication and tunes their allocation [42]. Cui et al. partition SMs between two persistent kernels for MoE [7]. ENTWINE shares SM resources between computation and communication, using a communication budget to balance communication progress against computation slowdown. ParallelKittens examines communication mechanisms and resource scheduling, including

overlap within an SM and across separate SMs [36]. HFUSE combines kernels with complementary resource demands through horizontal fusion [15]. Cui and Pericàs shape computation kernel residency and raise communication stream priority [6]. FiCCO characterizes decomposition overhead and resource contention, and uses DMA offloading to improve overlap [26]. ACE, ARK, and T3 reduce communication demands on compute resources through hardware support [11, 29, 33], while NVSHMEM and MSCCL++ expose lower-level transfer and synchronization primitives [10, 25].

## 7 Conclusion

Effective overlap requires computation to produce communicable tiles at a steady rate and communication to process them promptly. The benefit of overlap must also outweigh the accompanying computation slowdown. ENTWINE addresses these coupled requirements through interleaved tile production, independent tile-level communication, and shared-SM execution with a communication-concurrency budget. Across representative tensor-parallel LLM workloads, ENTWINE achieves a geomean speedup of 1.232 $\times$  (up to 1.433 $\times$ ) over cuBLAS+NCCL, and outperforms state-of-the-art overlap baselines by 3.1–9.8% in geomean.

## Acknowledgments

ChatGPT was used to assist with manuscript preparation, including language editing and  $\LaTeX$  formatting. All resulting content was reviewed and approved by the authors.

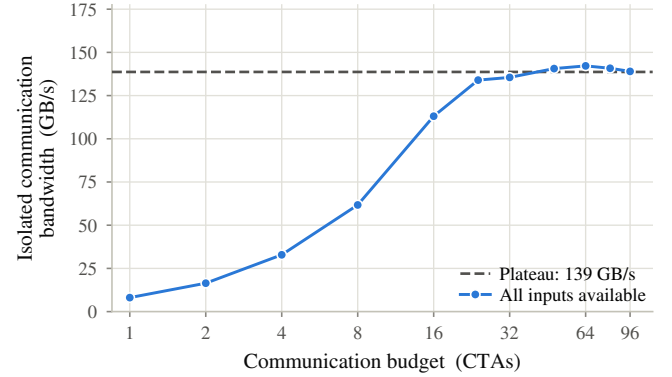
## References

- [1] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing Optimal Collective Algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. Association for Computing Machinery, New York, NY, USA, 62–75. doi:10.1145/3437801.3441620
- [2] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-Based Communication Overlap On GPUs Through Kernel Fusion. arXiv:2406.06858 doi:10.48550/arXiv.2406.06858
- [3] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3*. Association for Computing Machinery, New York, NY, USA, 178–191. doi:10.1145/3620666.3651379
- [4] Zhaodong Chen, Andrew Kerr, Richard Cai, Jack Kosaian, Haicheng Wu, Yufei Ding, and Yuan Xie. 2024. EVT: Accelerating Deep Learning Training with Epilogue Visitor Tree. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3*. Association for Computing Machinery, New York, NY, USA, 301–316. doi:10.1145/3620666.3651369
- [5] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*. Association for Computing Machinery, New York, NY, USA, 502–514. doi:10.1145/3575693.3575724
- [6] Minyu Cui and Miquel Pericàs. 2026. Resource-Aware Computation-Communication Overlap for Multi-GPU ML Workloads. Accepted at the AI on HPC Workshop at ISC 2026; arXiv version cited. arXiv:2606.09200 doi:10.48550/arXiv.2606.09200
- [7] Minyu Cui, Anna Wingkvist, and Morgan Ericsson. 2026. Fine-Grained Computation-Communication Overlap via Tile-Level Signaling and Scheduling for Mixture-of-Experts. Accepted at the 55th International Conference on Parallel Processing (ICPP 2026); arXiv version cited. arXiv:2607.19539 doi:10.48550/arXiv.2607.19539
- [8] Horace He, Less Wright, Luca Wehrstedt, Tianyu Liu, and Wanchao Liang. 2024. [Distributed w/ TorchTitan] Introducing Async Tensor Parallelism in PyTorch. PyTorch Developer Forums. Accessed 6 September 2026. <https://discuss.pytorch.org/t/distributed-w-torchtitan-introducing-async-tensor-parallelism-in-pytorch/209487>
- [9] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2026. Efficient and Adaptable Overlapping for Computation and Communication via Signaling and Reordering. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. Association for Computing Machinery, New York, NY, USA, 1894–1911. doi:10.1145/3767295.3769370
- [10] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*. Association for Computing Machinery, New York, NY, USA, 1201–1215. doi:10.1145/3779212.3790188
- [11] Changho Hwang, Kyoungsoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. 2023. ARK: GPU-Driven Code Execution for Distributed Deep Learning. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 87–101. <https://www.usenix.org/conference/nsdi23/presentation/hwang>
- [12] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 402–416. doi:10.1145/3503222.3507778
- [13] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of Machine Learning and Systems*, Vol. 5. MLSys, 341–353. [https://proceedings.mlsys.org/paper\\_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html](https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html)
- [14] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *The Ninth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=qrwe7XHTmYb>
- [15] Ao Li, Bojian Zheng, Gennady Pekhimenko, and Fan Long. 2022. Automatic Horizontal Fusion for GPU Kernels. In *Proceedings of the 2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 14–27. doi:10.1109/CGO53902.2022.9741270

- [16] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, Sanket Purandare, Gokul Nadathur, and Stratos Idreos. 2025. TorchTitan: One-Stop PyTorch Native Solution for Production Ready LLM Pretraining. In *The Thirteenth International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=SFN6Wm7YBI>
- [17] Daniel Lustig, Sameer Sahasrabudhe, and Olivier Giroux. 2019. A Formal Analysis of the NVIDIA PTX Memory Consistency Model. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 257–270. doi:10.1145/3297858.3304043
- [18] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better Together: Jointly Optimizing ML Collective Scheduling and Execution Planning Using SYNDICATE. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 809–824. <https://www.usenix.org/conference/nsdi23/presentation/mahajan>
- [19] Meta AI. 2025. Llama 3 and Llama 3.1 Model Configurations. Model artifact, commit 0e0b8c5. [https://github.com/meta-llama/llama-models/blob/0e0b8c5/models/sku\\_list.py](https://github.com/meta-llama/llama-models/blob/0e0b8c5/models/sku_list.py)
- [20] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3458817.3476209
- [21] NVIDIA. 2026. cuBLAS Library. NVIDIA Documentation. Accessed 3 September 2026. <https://docs.nvidia.com/cuda/cublas/>
- [22] NVIDIA. 2026. CUDA Programming Guide. NVIDIA Documentation. Accessed 3 September 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>
- [23] NVIDIA. 2026. CUTLASS: CUDA Templates for Linear Algebra Subroutines. Software, commit 08185b9c. <https://github.com/NVIDIA/cutlass/tree/08185b9c>
- [24] NVIDIA. 2026. NVIDIA Collective Communications Library (NCCL). NVIDIA Developer. Undated web page; 2026 denotes the version accessed on 9 September 2026. <https://developer.nvidia.com/nccl>
- [25] NVIDIA. 2026. NVSHMEM. NVIDIA Developer. Undated web page; 2026 denotes the version accessed on 9 September 2026. <https://developer.nvidia.com/nvshmem>
- [26] Shagnik Pal, Shaizeen Aga, Suchita Pati, Mahzabeen Islam, and Lizy K. John. 2025. Design Space Exploration of DMA Based Finer-Grain Compute Communication Overlap. arXiv:2512.10236 doi:10.48550/arXiv.2512.10236
- [27] Jason Jong Kyu Park, Yongjun Park, and Scott Mahlke. 2017. Dynamic Resource Management for Efficient Utilization of Multitasking GPUs. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Association for Computing Machinery, New York, NY, USA, 527–540. doi:10.1145/3037697.3037707
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [29] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2. Association for Computing Machinery, New York, NY, USA, 1146–1164. doi:10.1145/3620665.3640410
- [30] Kishore Punniyamurthy, Khaled Hamidouche, and Bradford M. Beckmann. 2024. Optimizing Distributed ML Communication with Fused Computation-Collective Operations. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 1–17. doi:10.1109/SC41406.2024.00094
- [31] Xinwei Qiang, Yue Guan, Zhengding Hu, Keren Zhou, Yufei Ding, and Adnan Aziz. 2026. Syncopate: Efficient Multi-GPU AI Kernels via Automatic Chunk-Centric Compute-Communication Overlap. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Seattle, WA, 331–347. <https://www.usenix.org/conference/osdi26/presentation/qiang>
- [32] Qwen Team. 2024. Qwen2.5-72B Model Configuration. Model artifact, revision efb10c. <https://huggingface.co/Qwen/Qwen2.5-72B/blob/efb10c/config.json>
- [33] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. 2021. Enabling Compute-Communication Overlap in Distributed Deep Learning Training Platforms. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 540–553. doi:10.1109/ISCA52012.2021.00049
- [34] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis Using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Boston, MA, 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [35] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. 2018. Mesh-TensorFlow: Deep Learning for Supercomputers. In *Advances in Neural Information Processing Systems*, Vol. 31. Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2018/hash/3a37abdeefe1dab1b30f7c5c7e581b93-Abstract.html>
- [36] Stuart H. Sul, Simran Arora, Benjamin F. Spector, and Christopher Ré. 2026. ParallelKittens: Systematic and Practical Simplification of Multi-GPU AI Kernels. In *Proceedings of Machine Learning and Systems*, Vol. 8. MLSys, 1076–1089. [https://proceedings.mlsys.org/paper\\_files/paper/2026/hash/ff997469ac66cf893c4183efeb22212a-Abstract-Conference.html](https://proceedings.mlsys.org/paper_files/paper/2026/hash/ff997469ac66cf893c4183efeb22212a-Abstract-Conference.html)
- [37] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973
- [38] Vasily Volkov and James W. Demmel. 2008. Benchmarking GPUs to Tune Dense Linear Algebra. In *Proceedings of the 2008 ACM/IEEE Conference on Supercomputing (SC)*. IEEE, 1–11. doi:10.1109/SC.2008.5214359
- [39] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. *Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping*. Technical Report MSR-TR-2024-40. Microsoft Research. <https://www.microsoft.com/en-us/research/publication/domino-eliminating-communication-in-llm-training-via-generic-tensor-slicing-and-overlapping/>

- [40] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2022. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1*. Association for Computing Machinery, New York, NY, USA, 93–106. doi:10.1145/3567955.3567959
- [41] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. arXiv:2105.04663 doi:10.48550/arXiv.2105.04663
- [42] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. COMET: Fine-Grained Computation-Communication Overlapping for Mixture-of-Experts. In *Proceedings of Machine Learning and Systems*, Vol. 7. ML-Sys. [https://proceedings.mlsys.org/paper\\_files/paper/2025/hash/e27ea0cd50b798ff8942caf9203f0992-Abstract-Conference.html](https://proceedings.mlsys.org/paper_files/paper/2025/hash/e27ea0cd50b798ff8942caf9203f0992-Abstract-Conference.html)
- [43] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Carlsbad, CA, 559–578. <https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin>
- [44] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. 2025. Triton-Distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. arXiv:2504.19442 doi:10.48550/arXiv.2504.19442
- [45] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels Using Tile-Centric Primitives. In *Proceedings of Machine Learning and Systems*, Vol. 7. MLSys. [https://proceedings.mlsys.org/paper\\_files/paper/2025/hash/c6ee784cbe46d854843e4c883a3321ef-Abstract-Conference.html](https://proceedings.mlsys.org/paper_files/paper/2025/hash/c6ee784cbe46d854843e4c883a3321ef-Abstract-Conference.html)

## A Communication Bandwidth in Isolation



**Figure 12.** Communication bandwidth with all inputs available and no competing producer. Figure 9 shows execution with concurrent computation.

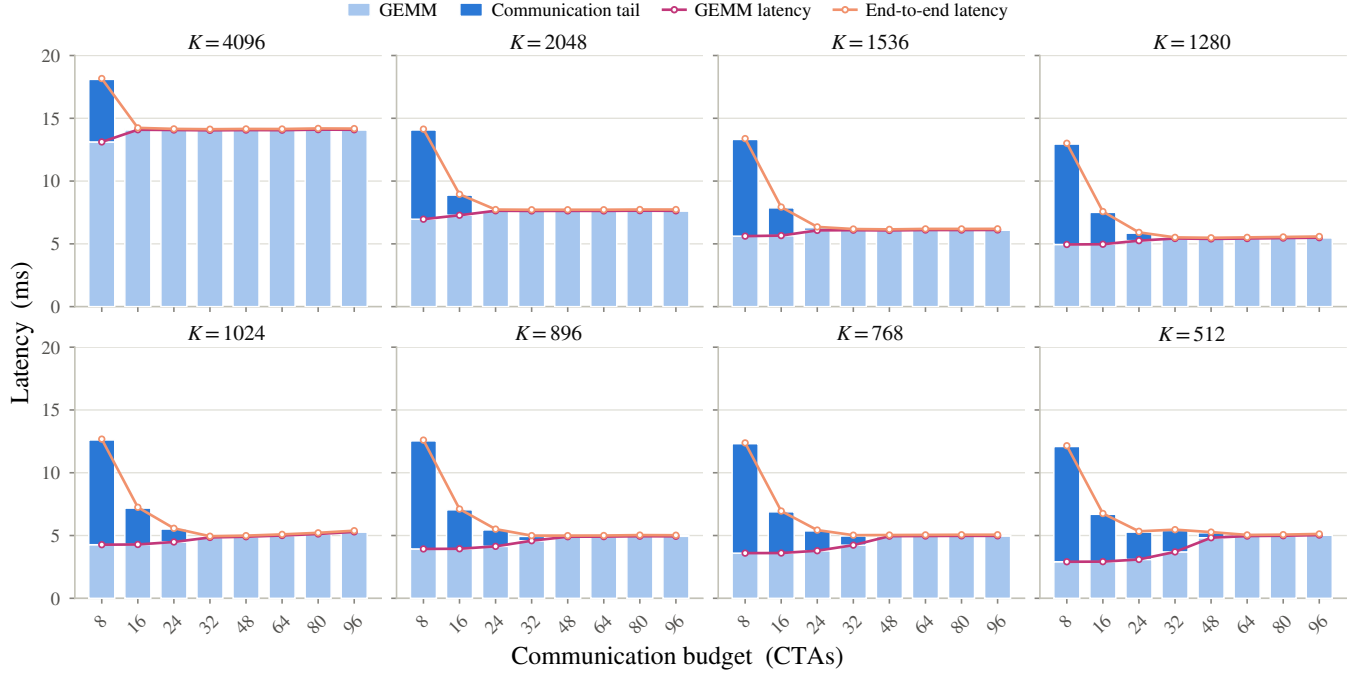
## B Full Communication-Budget Sweep

Figure 13 gives the full sweep discussed in §5.4.2, including the two cases in Figure 2 and the intermediate case in Figure 9.

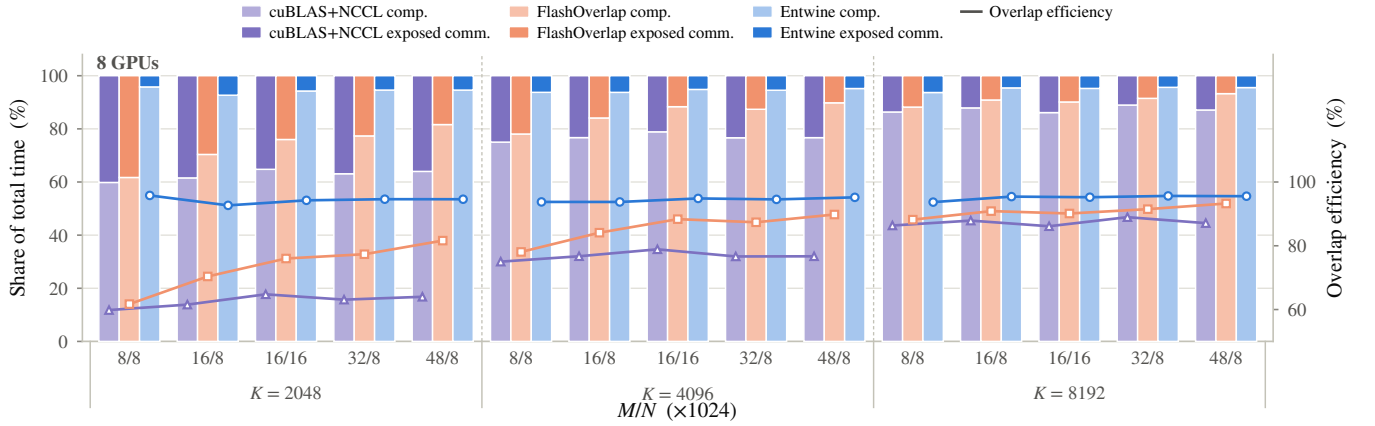
## C Overlap Efficiency Across Workloads

Figure 14 supplements the aggregate overlap efficiency in §5.2.1 with per-workload results. For each method, overlap efficiency is the ratio of uncontended producer time to end-to-end operator time. The remaining fraction includes the cost of communication and any producer slowdown relative to that baseline. It is therefore distinct from the measured communication tail in Figure 13.

On all 15 workloads, ENTWINE achieves higher overlap efficiency than cuBLAS + NCCL and FlashOverlap. Its efficiency remains high across the three reduction dimensions, while both baselines improve more substantially as  $K$  increases.



**Figure 13.** Coupled execution across communication budgets. Budgets cap concurrent consumer work. Bars show computation windows and communication tails; curves show GEMM and end-to-end latency, including completion overhead. Panels vary local  $K$  with  $M$  and  $N$  fixed. Figure 9 highlights the  $K = 768$  case.



**Figure 14.** Overlap efficiency. On the left axis, light bar segments show overlap efficiency. Dark segments show the remaining fraction. Curves repeat overlap efficiency on the right axis to show its trend; the two axes use different ranges. Each method uses its own uncontended producer as the computation baseline.